

A DDPG-based solution for optimal consensus of continuous-time linear multi-agent systems

Ye LI^{1,2}, Zhongxin LIU^{1,2*}, Ge LAN³, Malika SADER^{1,2} & Zengqiang CHEN^{1,2}¹College of Artificial Intelligence, Nankai University, Tianjin 300350, China;²Tianjin Key Laboratory of Intelligent Robotics, Nankai University, Tianjin 300350, China;³College of Software, Nankai University, Tianjin 300350, China

Received January 11, 2017; accepted April 6, 2017; published online January 1, 2017

Citation: Li Y, Liu Z X, Lan G, et al. A DDPG-based solution for optimal consensus of continuous-time linear multi-agent systems. Sci China Tech Sci, 2019, 62, <https://doi.org/>

Modeling a system in engineering applications is a time-consuming and labor-intensive task, as system parameters may change with temperature, component aging, etc. In this paper, a novel data-driven model-free optimal controller based on Deep Deterministic Policy Gradient (DDPG) is proposed to address the problem of continuous-time leader-following multi-agent consensus. In order to deal with the problem of the dimensional explosion of state space and action space, two different types of neural nets are utilized to fit them instead of the time-consuming state iteration process. With minimal energy consumption, the proposed controller achieves consensus only based on the consensus error and doesn't require any initial admissible policies. Besides, the controller is self-learning, which means it can achieve optimal control by learning in real time as the system parameters change. Finally, the proofs of convergence and stability, as well as some simulation experiments, are provided to verify the algorithm's effectiveness.

Leader-following Consensus, Optimal Control, Reinforcement Learning, Deep Deterministic Policy Gradient (DDPG)

1 Introduction

Multi-agent systems (MASs) are distributed systems consisting of individuals, communicating among them and their environment, which aim to complete some complex tasks. And they are widely used in intelligent robotics [1], traffic control [2], logistics scheduling [3], games [4], internet of things [5], and other fields.

Among the tasks of MASs, the consensus problem is one of the most fundamental problems in complex network systems, which has received a great deal of attention. In general, it can be divided into two sub-problems: leaderless consensus and leader-following consensus. In the case of a leaderless system [6–8], each agent tends

to agree on some indeterminate state. In the leader-following system [9–13], the following agents' states should eventually be the same as the leader's. At the same time, people seek to meet specific performance indicators such as energy consumption [14], efficiency [15], accuracy [16], and so on, which are known as optimal consensus problems. It should be noted that, however, the preceding results are predicated on the premise that the system model is known. System modeling, on the other hand, can be difficult and time-consuming in some cases, and the difficulty increases as the system complexity grows, making it a disaster for swarm intelligent systems. Besides, model parameters may vary from the age of the electronic items, temperature, humidity, magnetic field, and other factors. As a result, research into data-

*Corresponding author (email: lzhx@nankai.edu.cn)

driven optimal consensus techniques has become more and more popular in recent years.

Reinforcement learning (RL) is a branch of artificial intelligence whose purpose is to identify the best policies through trial and error, data collection, and experience learning [17]. These methods are beneficial for realizing model-free optimal control since they don't require all of the information about the system and simply build optimal solutions based on observations [18,19]. In 1991, Werbos introduced an adaptive dynamic programming (ADP) approach based on RL to find the best policy for discrete systems, and this policy iteration (PI) method only requires partial messages from the system [20]. Then, Doya expanded this technology to a continuous system in 2009 [21]. Lewis et al. offered an online solution to the optimal tracking control problem utilizing integral RL (IRL) [22,23]. So far, RL methods have been widely used in various fields such as constrained optimal control [24], time-delay optimal control [25], optimal tracking control [26], zero-sum game [28] and so on. In addition, it also has some preliminary applications in MASs [27]. For example, in [29], ADP methods were used to obtain the best control for graphical games. The consensus problem of discrete systems in [30,31] was solved using an adaptive learning framework and a strategy iteration method. The RL methods based on the neural network in [32] were used to address a class of time-delay nonlinear multi-agent consensus problems. Li et al. have presented a method based on Q-learning to establish agreement in discrete-time systems, where the information of environment is fully unknown [33–35]. Based on the preceding work, Xiang et al. extended this method to second-order nonlinear leader-following MASs [36].

However, most of the methods mentioned above are for discrete-time MASs, and two major issues still remain: (1) The application of most model-free methods is severely limited since they require initial admissible control policies, which are difficult to obtain. (2) The state space and action space grow in dimension as the number of agents or the range of states increases. Traditional model-free methods are no longer practical due to the inability to traverse every state space and action space multiple times or to calculate high-dimensional matrices.

Lillicrap et al. proposed the DDPG algorithm in 2016 [37], which solves the dimensional explosion and small space problem. So we propose a consensus solution for general continuous-time linear MASs based on it. Two actor nets and two critic nets are used to construct optimal policies and evaluate the value of the state-action pair, respectively. Besides, a value function based on the consensus error is designed to ensure the correct op-

timization direction of the nets. The proposed method is suitable for real systems like microgrid MASs and multi-satellite systems. Real-time optimal control can be achieved even for the system parameters may change due to the effect of the environment. In summary, the followings are the main contributions:

1. The deep RL methods are extended to the consensus problem of the model-free continuous-time leader-following MASs. The discrete data gained by sampling the continuous-time MASs is utilized to train nets, and trained actor nets can achieve optimal consensus control. By this mean, we provide a feasible idea to the continuous-time MASs control tasks.
2. The proposed method doesn't need initial admissible policies. In contrast to traditional model-free methods, our method doesn't require any initial admissible policies to generate valid learning data or enter valid learning states. Instead, only a set of policies with small initial values is required.
3. The proposed controller is highly flexible and strongly robust. A variable of adjustable parameters of the controller can satisfy different demands of performances, such as learning ability, control accuracy, system stability, and so on. Besides, when the system's inherent parameters change owing to temperature changes, parts aging, and other factors, the controller has the ability to self-learn and achieve optimal consensus control.

The structure of the paper is as follows: Several basic mathematical concepts are introduced in Section 2, then the process of building the controller is shown in Section 3. In Section 4, the convergence and stability of the algorithm are discussed to ensure that the proposed algorithm can get the optimal policies in theory. Finally, some simulation examples are provided to demonstrate the algorithm's training process and control effects in Section 5.

Notations: For matrix A , $\lambda_{\max}(A)$ represents the largest eigenvalue and $\lambda_{\min}(A)$ represents the smallest eigenvalue of it. I_N denotes the N -dimensional identity matrix and 1_N denotes the column vector with N elements being 1. The L_2 -norm of a vector m is denoted as $\|m\|$. $\otimes$ is the Kronecker product, ∇ is the gradient operator. A matrix with the matrices $a_1, a_2, \dots, a_n$ on its diagonal is denoted as $\text{diag}\{a_1, a_2, \dots, a_n\}$.

2 Problem Statement

2.1 Algebraic Graph Theory

Let $G = (V, E, A)$ be a directed weighted connected graph, where $V = \{v_1, v_2, \dots, v_n\}$ is a set containing N agents, $E \subseteq V \times V$ represents the set of edges and $I = \{1, 2, \dots, N\}$ represents the agent subscript set. A represents the nonnegative adjacency matrix of the graph G , with the elements representing the connection weights between agents: $a_{ij} > 0$ when agent i can receive information from agent j , otherwise $a_{ij} = 0$. It is critical to note that the topological network discussed in this paper is a simple digraph, meaning that $a_{ii} = 0, \forall i, j \in I$ and the information flow is directed. Define the neighbor set $N_i = \{j|v_j : (v_j, v_i) \in E, a_{ij} \neq 0\}$ for each agent i , and the number of the neighbors is defined as $|N_i|$. The degree matrix $D = \text{diag}\{d_i, i = 1, 2, \dots, N\}$ is made up of $d_i = \sum_{j=1}^{|N_i|} a_{ij}$. The in-degree and out-degree for each agent i are designed as $d_{in}(v_i) = \sum_{j=1}^{|N_i|} a_{ji}$ and $d_{out}(v_i) = \sum_{j=1}^{|N_i|} a_{ji}$, respectively. Define the Laplace matrix as $L = D - A$. If any agent i can reach any agent j through at least one directed path, the graph G is called strongly connected, which means that for every two agents i and j in the graph G , an ordered set of subscript $\{k_1, k_2, \dots, k_m\}$ exists, and $a_{ik_1} > 0, a_{k_1k_2} > 0, \dots, a_{k_mk_j} > 0$. If an agent in the graph G can reach every other agent by a directed path, the graph is said to contain a spanning tree, and this agent is known as the root agent.

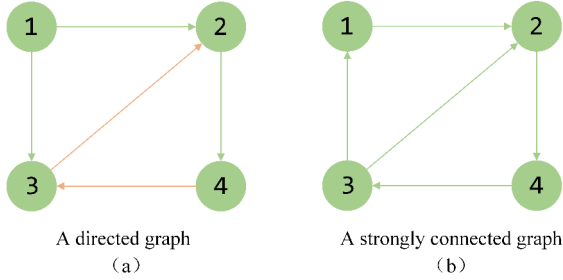


Figure 1 Two examples of connected graphs

2.2 LQR Problem

Consider a continuous-time MAS with $N + 1$ agents, the dynamic equation of the following agent i is described as follows:

$$\dot{x}_i = Ax_i + Bu_i, \quad (1)$$

where $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{n \times m}$ are the coefficient matrices for all the following agents. $x_i \in \mathbb{R}^n$ is the state of agent i and $u_i \in \mathbb{R}^m$ is the control policy of agent i . The state equation for the leader, which can generate the trajectory, is as follows:

$$\dot{x}_0 = Ax_0, \quad (2)$$

where $x_0 \in \mathbb{R}^n$ denotes the state of the leader.

Remark 1 The proposed solution is only applicable to MASs whose kinetic equations like (1) and (2). For heterogeneous MASs and practical MASs, two types of nets and the value function should be redesigned to ensure the availability and safety of the proposed solution.

The goal of this paper is to design a controller, which can drive each following agent i in the system to track the leader in real time. It means that if the tracking error between each following agent and leader tends to 0, the system will reach consensus, and the tracking error is defined as:

$$\eta = x - \bar{x}_0, \quad (3)$$

where $x = [x_1^T, x_2^T, \dots, x_n^T]^T$ is the state vector of following agents, $\bar{x}_0 = 1_N \otimes x_0$ is the leader state vector.

Since each agent in MASs can only obtain information from its surrounding neighbors, the tracking error (3) cannot be utilized to design the policies by all the following agents. However, if the error between each agent and its neighbors approaches 0, the system will reach consensus too. Thus the consensus error for each agent is defined as follows:

$$\varepsilon_i = \sum_{j=1}^{|N_i|} a_{ij}(x_j - x_i) + g_i(x_0 - x_i), \quad (4)$$

where a_{ij} is the connection weight in the adjacency matrix A , and g_i is the connection weight between agent i and the leader: if agent i can receive message from the leader, $g_i = 1$, otherwise $g_i = 0$. The connection matrix is defined as $\Gamma = \text{diag}\{g_i, i = 1, 2, \dots, N\}$. The consensus error for the whole system can be represented as follows:

$$\varepsilon = -(L + \Gamma)x + \Gamma\bar{x}_0, \quad (5)$$

where $\varepsilon = [\varepsilon_1^T, \varepsilon_2^T, \dots, \varepsilon_n^T]^T$ is the global consensus error vector.

In order to guarantee the existence of the stabilizing policy, the following assumptions are needed:

Assumption 1 The inherent matrices A and B of the system (1) and (2) are determined and unknown.

Assumption 2 The system matrix pair (A, B) is controllable.

Assumption 3 The system communicating net topology contains a spanning tree, and at least one following agent is linked to the leader.

Lemma 1 [30] If matrix $L + \Gamma$ is non-singular, then the following equation holds:

$$\|\eta\| \leq (\lambda_{\min}(L + \Gamma))\|\varepsilon\|. \quad (6)$$

If the model is known, traditional optimal control methods will solve the problem perfectly: the policy can be designed based on consensus error by using feedback control principle [30]:

$$u = cK\varepsilon, \quad (7)$$

where c is the coupling gain, K is the feedback matrix, both of which can be designed by system and topology information. Then the closed-loop dynamic can be written as:

$$\dot{x} = (I_N \otimes A - c(L + G) \otimes BK)x + (c(L + G) \otimes BK)\bar{x}_0. \quad (8)$$

However, obtaining the system information is always challenging: what we get directly is part of the system states. The matrices A and B can be estimated using system identification technology such as system observer, but they are not always precise and are costly. Recently, some researches have used Q-learning and its variants to generate control policies that partially solve related issues in discrete-time systems. They calculate the value of each action in each state and choose the most valuable action of the current state, and optimal control can be achieved by continuous iterations.

Unfortunately, the state space and action space of discrete-time MASs expand as the state range or the number of agents expands. Traditional model-free RL methods perform bad due to inability to compute high-dimensional matrices [33, 34] or to traverse all spaces multiple times.

2.3 Nash Equilibrium

Since each agent in MASs can only get data through their neighbors, their actions change the environment and increase the system's instability. So the following lemmas are required.

Lemma 2 [38] **Admissible control:** If the policy $u(t)$ can not only stabilize the system (1) and (2) but also bound the tracking error (3), it is called an admissible control.

Lemma 3 [38, 39] **Global Nash Equilibrium:** If any agent chooses the optimal policy when all other agents' policies have been chosen, this combination is known as the Nash equilibrium, and the following inequality holds:

$$\begin{aligned} V_i^* &= V_i(u_1^*, u_2^*, \dots, u_i^*, \dots, u_N^*) \\ &\leq V_i(u_1^*, u_2^*, \dots, u_i, \dots, u_N^*). \end{aligned} \quad (9)$$

3 DDPG-based controller for Leader-following Multi-agent Consensus

3.1 Algorithm framework

In order to be consistent with the field of RL, both a_t and u_t in this paper represent the control input at time t . According to traditional methods, the consensus error can be used to obtain optimal control policies. Based on RL methods, neural networks can be used to avoid dimensional explosion. As a result, the consensus error (4) will be utilized as the input of the actor network $\mu(\varepsilon|\theta^\mu)$. Then a set of control policies of N following agents will be obtained after the network has been fitted over time. Since the state-action pairs are also high dimensional information, determining its value involves the use of a critic net $Q(\varepsilon, a|\theta^Q)$. So the goal of actor net is to increase the state-action pair's value, whereas the goal of critic net is to predict the true value of the state-action pair.

Only 2 neural nets, on the other hand, will cause 2 severe issues: 1. The data is not independent and identically distributed, which is essential for neural network convergence. 2. The bootstrapping problem: it can result in over-estimates or under-estimates of policies. The solution to the first problem is given by The Experience Replay Buffer technology [40]: when the actor interacts with the environment, store the feedback information in the buffer, shuffle the order randomly, and extract a fixed batch BS from it during the training progress. In this way, the data will approximate the independent and identically distributed state to some extent. When it comes to the second issue, Target Net technology [37] can help. Target nets have the same network structure as the value nets but different parameters. When the nets are updated, the target nets will generate the value of state-action pair and the next-time policies, which are utilized for value net iteration rather than system control. The result of target nets will be referred to as $\bar{\mu}(\varepsilon|\bar{\theta}^\mu)$ and $\bar{Q}(\varepsilon, a|\bar{\theta}^Q)$.

Remark 2 The discrete data stored in the replay buffer is collected by sampling the MAS (1) and (2), and it will be used to train the nets. Finally, the trained actor nets can be utilized to control the continuous-time MASs.

According to the global Nash Equilibrium theorem, the state loss L is defined as the negative mean sum of squares of each agent's tracking error, and the equation is as follows:

$$L = -\frac{[\varepsilon_1^T, \varepsilon_2^T, \dots, \varepsilon_n^T] \cdot [\varepsilon_1^T, \varepsilon_2^T, \dots, \varepsilon_n^T]^T}{N} = -\frac{\varepsilon\varepsilon^T}{N}. \quad (10)$$

Since DDPG uses the gradient ascent algorithm to improve net performance by enhancing the state-action

value, we define the return as the negative loss.

3.2 Actor net and Critic net

The proposed controller consists of actor nets μ and critic nets Q , each part includes a value net and a target net. The value critic net is actually a BP net, which consists of n_c layers, $critic_input = [\varepsilon_1, \varepsilon_2, \dots, \varepsilon_n, a_1, a_2, \dots, a_n]^T$ is the state-action pair composed of consensus error and policies vector, the output is the value of the state-action pair. The structure can be written as:

$$\begin{cases} H_1 = f(W_1 critic_input + B_1) \\ H_2 = f(W_2 H_1 + B_2) \\ \vdots \\ Y = f(W_{n_c} H_{n_c-1} + B_{n_c}) \end{cases}, \quad (11)$$

where $f(\cdot)$ is the Relu activation function, H_i is the hidden layer, W_i is the coefficient matrix of the net, B_i is the bias matrix. $i = 1, 2, \dots, n_c$ is the layer number of the critic net, and n_c is the total layer number of critic net. We call the value net as $Q(\varepsilon, a|\theta^Q)$, when constructing the value Q net, the target Q net is constructed as well, with the same structure and parameters. Then the estimated Q value at the time t is $q_t = Q(\varepsilon_t, a_t|\theta^Q)$, the action and value predicted by target net at $t + 1$ are $q'_{t+1} = \tilde{Q}(\varepsilon_{t+1}, \mu'_{t+1}|\theta^{\tilde{Q}})$, $\mu'_{t+1} = \tilde{\mu}(\varepsilon_{t+1}|\theta^{\tilde{\mu}})$, and the Temporal-Difference target ($TD - target$) is:

$$y_t = r_t + \gamma \tilde{Q}(\varepsilon_{t+1}, \mu'_{t+1}|\theta^{\tilde{Q}}), \quad (12)$$

where r_t is the return at time t , $\mu(\varepsilon|\theta^\mu)$ is the actor net. Traditional Mean-Square error (MSE error) is used to optimize the net:

$$L_c = \frac{1}{N_b} \sum_i ((r_t + \gamma \tilde{Q}(\varepsilon_{t+1}, \mu'_{t+1}|\theta^{\tilde{Q}}) - Q(\varepsilon_t, a_t|\theta^Q))^2, \quad (13)$$

where N_b is the batch size of training data.

The actor net is also a BP net with the consensus error ε_i as its input and the action vector as its output. The following is the structure:

$$\begin{cases} H_1 = f(W_1 \varepsilon_i + B_1) \\ H_2 = f(W_2 H_1 + B_2) \\ \vdots \\ out = \tanh(W_{n_a} H_{n_a-1} + B_{n_a}) \\ Y = out \times a_u \end{cases}, \quad (14)$$

where $\tanh(\cdot)$ is the tanh activation function, and n_a is the total layer number of the actor net. In contrast to the critic net, when generating the action vector, the

output will be normalized first, then enlarged to the size required by the system to avoid generating an excessively large action vector. The system actor's upper bound is a_u .

Remark 3 The selection of the upper bound is critical. When constructing the controller for the real MASs, the action space should be built as large as possible under the premise of training safety. In this way, the actor net can learn optimal policies both from convergent and divergent trajectories and find the correct optimization direction.

The Deterministic policy significantly reduces the calculations and increases the speed of net convergence. However, the lack of exploration leads to severe local optimization [37], therefore we add a gaussian noise on it, resulting in the final policy:

$$a_t = a_t + g\left(\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right)\right), \quad (15)$$

where σ is the variance controlling the exploration rate and function g is a sampling function. The variance is calculated as follows:

$$\sigma = \sigma_u - i_e \cdot \frac{\sigma_u}{N_e} + 0.001, \quad (16)$$

where i_e is the current episode number, N_e is the total episode number, and σ_u is the upper bound of the variance. The purpose of continuous iteration is to identify appropriate policies to maximize $Q(\varepsilon, a|\theta^Q)$, that is, to maximize the following equation:

$$J_\beta(\mu^\theta) = \int_E \rho^\beta(\varepsilon) Q(\varepsilon, \mu^\theta(\varepsilon)) d\varepsilon, \quad (17)$$

where E is the set of consensus errors in all case, $\rho^\beta(\varepsilon)$ is the distribution function of the state, its gradient is:

$$\nabla_{\theta^\mu} J_\beta(\mu) \approx E_{\varepsilon \rightarrow \rho^\beta} [\nabla_a Q(\varepsilon, \mu^\theta(\varepsilon))|_{a=\mu(\varepsilon)} \nabla_{\theta^\mu} \mu(\varepsilon|\theta^\mu)]. \quad (18)$$

According to the Monte-Carlo method [41], an un-biased estimate can be obtained by using mini-batch data, thus the equation can be rewritten as follows.

$$\nabla u = \nabla_a Q(\varepsilon, \mu^\theta(\varepsilon))|_{\varepsilon=\varepsilon_i, a=\mu(\varepsilon_i)} \nabla_{\theta^\mu} \mu(\varepsilon|\theta^\mu)|_{\varepsilon=\varepsilon_i}, \quad (19)$$

$$\nabla_{\theta^\mu} J_\beta(\mu) \approx \frac{1}{N_b} \sum_i \nabla u, \quad (20)$$

where ∇u is the single return gradient. Then the value $\mu(\varepsilon|\theta^\mu)$ net will be updated by gradient boosting algorithm. The detailed description of each net is shown in Table 1 and the flowchart is shown in Fig.2.

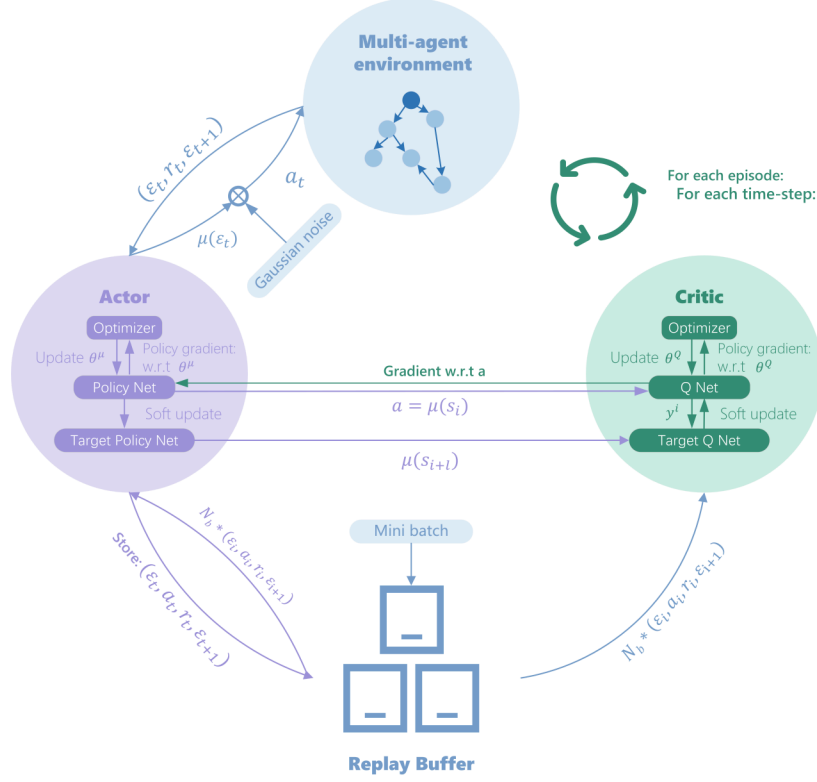


Figure 2 The algorithm flow chart in the model-free leader-following multi-agent consensus problem

3.3 Algorithm flow

A detailed description of the algorithm will be given in this section, as well as various tricks in the training process that play an essential role in our solution.

- Initialize 2 value nets and 2 target nets, and assign the parameters θ of value nets to target nets.
- Initialize the replay memory buffer R .
- Starting training, entering the first layer of loop, for each epoch:

1. Initialize the environment. The initial state of the agents should be different in each epoch so that more information can be used to optimize the nets.
2. Initialize the Gaussian noise. Start the second loop until the system diverges or reaches the max time $t_{\max}$:
 - (a) A set of policies a_t is generated by the actor net and acted on the environment.
 - (b) Store the feedback information in the replay buffer R . For continuous-time MASSs, a threshold λ can be set. If the change of state is greater than λ , then store it. This will reduce the amount of worthless information, which will improve training.
 - (c) Calculate a'_{t+1} and q'_{t+1} through target nets, then update value critic net.

- (d) Calculate the gradient of the reward to the value action net parameters and update the value actor net.
- (e) Update target nets by the moving average method [37], which is essential for the stability of the nets. Generally, τ is a small positive number such as 0.001:

$$\begin{cases} \theta^Q = \tau \theta^Q + (1 - \tau) \theta^Q \\ \theta^\mu = \tau \theta^\mu + (1 - \tau) \theta^\mu \end{cases} \quad (21)$$

End the inner loop.

End the outer loop.

Unlike classical observer methods, our algorithm has the ability to learn in real time. When the system changes owing to parts aging, temperature impacts, mechanical looseness, voltage fluctuations, and other factors, the information can update in real time, all we have to do is to enable training mode when the consensus error exceeds a particular level.

4 Proofs of Convergence and Stability

In this section, the proofs of the convergence and stability of the proposed controller is given. For critic net, the *TD-target* (12) is used as the target for gradient ascent,

Table 1 The differences of 4 nets

Algorithm	Neural Network	Input	Output
Critic	target net $\tilde{Q}(\varepsilon_{t+1}, \tilde{\mu}(\varepsilon_{t+1} \theta^{\tilde{\mu}}) \theta^{\tilde{Q}})$	(1) The consensus error of next time ε_{t+1} . (2) The policies made by target net $\tilde{\mu}$.	Output $\tilde{Q}$ used to calculate the target reward. The equation is: $y_t = r_t + \gamma \tilde{Q}(\varepsilon_{t+1}, \tilde{\mu}(\varepsilon_{t+1} \theta^{\tilde{\mu}}) \theta^{\tilde{Q}})$
	Critic net $Q(\varepsilon_t, a_t \theta^Q)$	(1) The consensus error of current time ε_t . (2) The actual action a_t .	Output the value of state-action pair. This value is used to calculate $TD - target$ and update the value actor net. $L_c = \frac{1}{N_b} \sum_t (y_t - Q(\varepsilon_t, a_t \theta^Q))^2$ $\nabla u = \nabla_a Q(\varepsilon, \mu^\theta(\varepsilon)) _{\varepsilon=\varepsilon_t, a=\mu(\varepsilon_t)} \nabla_{\theta^\mu} \mu(\varepsilon \theta^\mu) _{\varepsilon=\varepsilon_t}$
Actor	Target net $\tilde{\mu}(\varepsilon_{t+1} \theta^{\tilde{\mu}})$	The consensus error of next time ε_{t+1} .	Output is the policy vector $\tilde{\mu}$. The vector is used to calculate the target net $\tilde{Q}$. $\tilde{Q}(\varepsilon_{t+1}, \tilde{\mu}(\varepsilon_{t+1} \theta^{\tilde{\mu}}) \theta^{\tilde{Q}})$
	Actor net $\mu(\varepsilon \theta^\mu)$	The consensus error of current time ε_t .	Output is the policy vector μ . The vector will be used to update the value actor net with the critic net Q . $\nabla u = \nabla_a Q(\varepsilon, \mu^\theta(\varepsilon)) _{\varepsilon=\varepsilon_t, a=\mu(\varepsilon_t)} \nabla_{\theta^\mu} \mu(\varepsilon \theta^\mu) _{\varepsilon=\varepsilon_t}$

the update process is as follows:

$$\theta_t^Q = \theta_t^Q + \alpha_\theta L_{c(t)} \nabla_\theta Q(\varepsilon_t, a_t|\theta^Q), \quad (22)$$

where α_θ is the learning rate of the net. The update direction is given by the gradient $\nabla_\theta Q(\varepsilon_t, a_t|\theta^Q)$ of Q net, and the step of the network update depends on the MSE error $L_{c(t)}$, so the iteration reward y_t directly affects the convergence of the net. The target net only delays the updating speed of the network but doesn't affect the convergence of the original system [37]. Thus, the following analysis will be based on the iteration reward y_t .

4.1 Proof of Convergence

Lemma 4 [37] For $\forall t \in I$ and $\forall r \in N$, the actor net $Q_t^r(\varepsilon_t, a_t|\theta^Q)$ and critic net $\mu_t^r(\varepsilon|\theta^\mu)$ are both updated by gradient ascent method. If each agent's initial strategies a_t^0 are admissible, then the iterative policies a_t^r are still admissible for $\forall r = 0, 1, 2, \dots$. And the policies can not only stabilize the system, but also limit the loss function, allowing the value function (10) to be maximized.

Lemma 5 For $\forall t \in I$ and $\forall r \in N$, if the initial policies a_t^0 are admissible, the critic net $Q(\varepsilon_t, a_t|\theta^Q)$ is monotonically non-decreasing.

$$Q_t^{r+1}(\varepsilon_t, a_t^r|\theta^Q) \geq Q_t^r(\varepsilon_t, a_t^r|\theta^Q). \quad (23)$$

Proof. The consensus error at time t is used to calculate the return:

$$r_t = L = -\frac{\varepsilon \varepsilon^T}{N},$$

Furthermore, the return at a later time is related not just to the current consensus error, but also to the policies in effect at the time, so the return is defined as $R_t^{r+1}(\varepsilon_t, a_t^{r+1})$. Then the iteration error at time t is:

$$\begin{aligned} y_t^{r+1}(\varepsilon_t, a_t^{r+1}) &= R_t^{r+1}(\varepsilon_t, a_t^{r+1}) \\ &\quad + \gamma Q_t^r(\varepsilon_{t+1}, \mu^r(\varepsilon_{t+1}|\theta^\mu)|\theta^Q) \\ &= \max_{a_t} R_t^r(\varepsilon_t, a_t^r) \\ &\quad + \gamma Q_t^r(\varepsilon_{t+1}, \mu^r(\varepsilon_{t+1}|\theta^\mu)|\theta^Q) \\ &\geq y_t^r(\varepsilon_t, a_t^r). \end{aligned}$$

In the iterative process, the policies $\mu(\varepsilon|\theta^\mu)$ are always admissible based on Lemma 4, so when $t = T$, $T \rightarrow \infty$, $R \rightarrow 0$, then

$$\begin{aligned} Q_T^{r+1}(\varepsilon_T, \mu^{r+1}(\varepsilon_T|\theta^\mu)|\theta^Q) &= y_T^{r+1}(\varepsilon_T, a_T^{r+1}) \\ &= Q_T^r(\varepsilon_T, \mu^r(\varepsilon_T|\theta^\mu)|\theta^Q). \end{aligned}$$

Let $t = T - 1$, then

$$\begin{aligned} y_{T-1}^{r+1}(\varepsilon_{T-1}, a_{T-1}^{r+1}) &= R_{T-1}^{r+1}(\varepsilon_{T-1}, a_{T-1}^{r+1}) \\ &\quad + \gamma Q_{T-1}^r(\varepsilon_T, \mu^r(\varepsilon_T|\theta^\mu)|\theta^Q) \\ &= R_{T-1}^{r+1}(\varepsilon_{T-1}, a_{T-1}^{r+1}) \\ &\quad + \gamma Q_{T-1}^{r-1}(\varepsilon_T, \mu^{r-1}(\varepsilon_T|\theta^\mu)|\theta^Q) \\ &= \max_{a_{T-1}} R_{T-1}^r(\varepsilon_{T-1}, a_{T-1}^r) \end{aligned}$$

$$\begin{aligned}
& + \gamma Q_{T-1}^{r-1}(\varepsilon_T, \mu^{r-1}(\varepsilon_T | \theta^\mu)) | \theta^\mathcal{Q} \\
& \geq y_{T-1}^r(\varepsilon_{T-1}, a_{T-1}^r).
\end{aligned}$$

For $\forall s = 0, 1, 2, \dots$, when $t = s + 1$,

$$y_{s+1}^{r+1}(\varepsilon_{s+1}, a_{s+1}^{r+1}) \geq y_{s+1}^r(\varepsilon_{s+1}, a_{s+1}^r).$$

Let $t = s$, then

$$\begin{aligned}
y_s^{r+1}(\varepsilon_s, a_s^{r+1}) &= R_s^{r+1}(\varepsilon_s, a_s^{r+1}) \\
&+ \gamma Q_s^r(\varepsilon_{s+1}^r, \mu^r(\varepsilon_{s+1} | \theta^\mu)) | \theta^\mathcal{Q} \\
&= \max_{a_s} R_s^r(\varepsilon_s, a_s^r) \\
&+ \gamma Q_s^r(\varepsilon_{s+1}, \mu^r(\varepsilon_{s+1} | \theta^\mu)) | \theta^\mathcal{Q} \\
&\geq y_s^r(\varepsilon_s, a_s^r).
\end{aligned}$$

As we can see, the iteration reward gradually increases with training, the Q function is monotonic and non-decreasing. So the inequality (23) holds.

Theorem 1 Considering the system (1) and (2), if the initial policies a_0^0 are admissible, the control policies are given by actor net (11), the value of the state-action pair are given by critic net (14), and the nets are updated by gradient ascent method, when $t \rightarrow \infty$, $Q_t^r(\varepsilon_t, a_t^r | \theta^\mathcal{Q})$ will converge to the optimal value $Q_t^*(\varepsilon_t, a_t^* | \theta^\mathcal{Q})$ and a_t^r will converge to the optimal policy a_t^* .

Proof. $y_t^r(\varepsilon_t, a_t^r)$ is a monotonic non-decreasing function according to Lemma 5, so the limit of this equation exists, and let the limit be $y_t^\infty(\varepsilon_t, a_t^\infty) = \lim_{r \rightarrow \infty} y_t^r(\varepsilon_t, a_t^r)$, then:

$$\begin{aligned}
y_t^\infty(\varepsilon_t, a_t^\infty) &\geq y_t^{r+1}(\varepsilon_t, a_t^{r+1}) \\
&= \max_{a_t} (R_t^r(\varepsilon_t, a_t^r) \\
&+ \gamma Q_t^r(\varepsilon_{t+1}^r, \mu^r(\varepsilon_{t+1} | \theta^\mu)) | \theta^\mathcal{Q}),
\end{aligned}$$

when $r \rightarrow \infty$,

$$\begin{aligned}
y_t^\infty(\varepsilon_t, a_t^\infty) &\geq \max_{a_t} (R_t(\varepsilon_t, a_t)) \\
&+ \gamma Q_t^\infty(\varepsilon_{t+1}^\infty, \mu^\infty(\varepsilon_{t+1} | \theta^\mu)) | \theta^\mathcal{Q}.
\end{aligned}$$

Since $y_t^r(\varepsilon_t, a_t^r)$ is monotonic and non-decreasing, then the iteration number R must exist satisfying the following inequality:

$$\begin{aligned}
y_t^\infty(\varepsilon_t, a_t^\infty) &\leq y_t^R(\varepsilon_t, a_t^R) - \xi \\
&= \max_{a_t} (R_t^{R-1}(\varepsilon_t, a_t^{R-1})) - \xi \\
&+ \gamma Q_t^{R-1}(\varepsilon_{t+1}, \mu^{R-1}(\varepsilon_{t+1} | \theta^\mu)) | \theta^\mathcal{Q} \\
&\leq \max_{a_t} (R_t(\varepsilon_t, a_t)) - \xi \\
&+ \gamma Q_t^\infty(\varepsilon_{t+1}^\infty, \mu^\infty(\varepsilon_{t+1} | \theta^\mu)) | \theta^\mathcal{Q},
\end{aligned}$$

where ξ is any real number, then let $\xi = 0$

$$y_t^\infty(\varepsilon_t, a_t^\infty) \leq \max_{a_t} (R_t(\varepsilon_t, a_t))$$

$$+ \gamma Q_t^\infty(\varepsilon_{t+1}^\infty, \mu^\infty(\varepsilon_{t+1} | \theta^\mu)) | \theta^\mathcal{Q}.$$

Thus

$$\begin{aligned}
y_t^\infty(\varepsilon_t, a_t^\infty) &= \max_{a_t} (R_t(\varepsilon_t, a_t)) \\
&+ \gamma Q_t^\infty(\varepsilon_{t+1}^\infty, \mu^\infty(\varepsilon_{t+1} | \theta^\mu)) | \theta^\mathcal{Q}.
\end{aligned} \tag{24}$$

Let $t = s$, when $s \rightarrow \infty$, the following inequality holds.

$$\begin{aligned}
y_s^r(\varepsilon_s, a_s^r) &= \max_{a_s} (R_s(\varepsilon_s, a_s)) \\
&+ \gamma Q_s^{r-1}(\varepsilon_{s+1}^{r-1}, \mu^{r-1}(\varepsilon_{s+1} | \theta^\mu)) | \theta^\mathcal{Q} \\
&\leq \max_{a_s} (R_s(\varepsilon_s, a_s)) \\
&+ \gamma Q_s^\infty(\varepsilon_{s+1}^\infty, \mu^\infty(\varepsilon_{s+1} | \theta^\mu)) | \theta^\mathcal{Q} \\
&= y_s^\infty(\varepsilon_s, a_s^\infty),
\end{aligned}$$

when $\forall s = 0, 1, 2, \dots$,

$$\begin{aligned}
y_s^r(\varepsilon_s, a_s^r) &= R_s^r(\varepsilon_s, a_s^r) \\
&+ \gamma Q_s^{r-1}(\varepsilon_{s+1}^{r-1}, \mu^{r-1}(\varepsilon_{s+1} | \theta^\mu)) | \theta^\mathcal{Q} \\
&\leq \max_{a_s} (R_s(\varepsilon_s, a_s)) \\
&+ \gamma Q_s^\infty(\varepsilon_{s+1}^\infty, \mu^\infty(\varepsilon_{s+1} | \theta^\mu)) | \theta^\mathcal{Q} \\
&= y_s^\infty(\varepsilon_s, a_s^\infty).
\end{aligned}$$

So the follow inequality holds:

$$y_t^r(\varepsilon_t, a_t^r) \leq y_t^\infty(\varepsilon_t, a_t^\infty), \forall t = 0, 1, 2, \dots \tag{25}$$

Let $a_t^r = a_t^*$, then according to (25), we can get

$$y_t^\infty(\varepsilon_t, a_t^\infty) \geq y_t^*(\varepsilon_t, a_t^*), \tag{26}$$

where $y_t^*(\varepsilon_t, a_t^*)$ is the iteration error when the system reach optimal state, so

$$y_t^\infty(\varepsilon_t, a_t^\infty) = \lim_{r \rightarrow \infty} y_t^r(\varepsilon_t, a_t^r) = y_t^*(\varepsilon_t, a_t^*). \tag{27}$$

Combine (26) and (27), and we get

$$y_t^\infty(\varepsilon_t, a_t^\infty) = y_t^*(\varepsilon_t, a_t^*). \tag{28}$$

According to gradient ascent method, when the system converges to the optimal performance $Q_t^*(\varepsilon_t, a_t^* | \theta^\mathcal{Q})$, the optimal policy a_t^* can be obtained at the same time.

4.2 Proof of Stability

Theorem 2 For MAS (1) and (2), if Assumptions 1-3 hold, then the system can reach consensus by the proposed policy (11) with the lowest energy consumption.

Proof. According to Lemma (5), the Q function is monotonic non-decreasing function, which means:

$$Q_t^{r+1}(\varepsilon_t, a_t^r | \theta^Q) - Q_t^r(\varepsilon_t, a_t^r | \theta^Q) \geq 0. \quad (29)$$

As we all know, the DDPG algorithm discretizes the value function of the MASs, and the nets are trained to maximize the value function. The actor net can achieve optimal control when the value function achieves its maximum value [37]. So all we have to do is to analyses the value function's properties, the Lyapunov difference function can be defined as:

$$\Delta L_t = - (Q_t^{r+1}(\varepsilon_t, a_t^r | \theta^Q) - Q_t^r(\varepsilon_t, a_t^r | \theta^Q)). \quad (30)$$

The Lyapunov function should take the opposite of the Q function difference because what we get is the negative loss. Based on inequality (29) and Theorem 1:

$$\Delta L_t \leq 0, \forall t = 0, 1, 2, \dots \quad (31)$$

It can be seen from the above analysis, when $t \rightarrow \infty$, $\varepsilon \rightarrow 0$, the system (1),(2) will reach consensus eventually and the energy sumption will be the lowest.

5 Simulation

Consider a system with the kinetic equations (1) and (2) consisting of a leader and 5 following agents. The system matrices are designed as:

$$A = \begin{bmatrix} -2 \pm 0.2\psi & -1 \pm 0.1\psi \\ 2 \pm 0.3\psi & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \pm 0.05\psi \\ \psi \end{bmatrix}$$

where $\psi \in [0, 1]$ is the environment factor, which can simulate the influence of the environment on the inherent parameters of the MASs. The Laplacian matrix L and the connection matrix Γ are presented below:

$$L = \begin{bmatrix} 2 & 0 & 0 & 0 & -2 \\ -1 & 4 & 0 & -3 & 0 \\ 0 & -1.5 & 1.5 & 0 & 0 \\ -2 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -4 & 4 \end{bmatrix}, \quad \Gamma = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The topology is shown in Fig.3. The initial value is randomly chosen within the range $[-10, 10]$, and the system is regarded to diverge if the absolute value of the states reaches 200. In order to accelerate the learning rate of the controller, the max absolute value of the control policy is set to 200. Experiments show that a sufficiently broad action space allows the system to learn the best policies quickly, while stability is degraded to some amount.

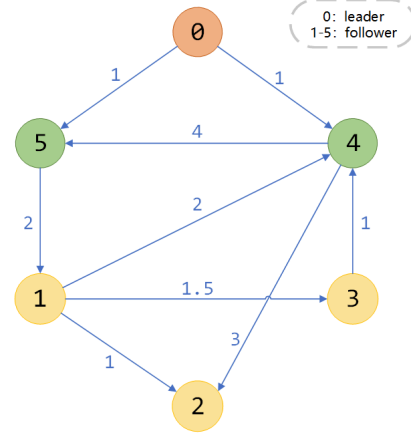


Figure 3 The system communication topology with a leader agent and 5 following agents

We build 2 actor nets and 2 critic nets for the controller. The actor nets contain 4 layers: one input layer, 2 hidden layers with 256 neurons, and one output layer with 5 neurons. The critic nets contain 4 layers: one input layer, 2 hidden layers with 256 neurons each, and one output layer with one neuron. And the hyperparameters are shown in table 2.

Table 2 The hyperparameter of the controller

Variable	Value	Variable	Value
max_state	200	max_time	7s
Buffer capacity	100000	record_time	0.05s
c_learning_rate	0.002	record_change	0.1
a_learning_rate	0.001	σ_u	0.02
a_u	200	N_e	100
N_b	64	γ	0.99
τ	0.005		

Remark 4 The proposed controller has a large number of customizable parameters, giving it greater flexibility. For example, raising *max_state* and a_u can broaden the learning range, improving robustness, but the controller's efficiency will decrease. Therefore, the values should be adjusted based on the actual situation.

As the controller iterates, the average return gradually increases and reaches 0, as shown in Fig.4. Since the controller knows nothing about the MASs during the first 7 iterations, the return declines rapidly. However, between 7 and 60 iterations, the return rises sharply, and the actor and critic nets learn a lot through the constant trial and error. The return then gradually increases, and the system tends to stabilize.

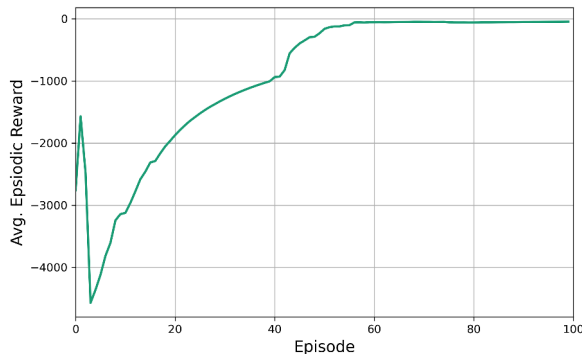


Figure 4 The graph of the system's average return evolution over time

The state evolution of agents under 4 different returns are depicted in Fig.5. The green line represents the leader's state, whereas the others are those of the following agents. As shown in the figure, the state of following agents shifts from chaos to order over time, and the final states tend to converge instead of diverge, indicating that the controller successfully bounded the states of all the agents. Furthermore, when the return increases, the system's steady tracking error decreases, implying that the system is learning more system information. When the return is about -35, the states of all following agents tend to that of the leader. Actually, the iteration speed is very fast at the start of training since the following agents always reach the max state before reaching the max time, causing the system to be judged divergent and the next cycle to begin. Every iteration can reach the max time when the controller has enough system information.

Fig.6 depicts the tracking error. Although each agent cannot acquire global information during the control process, analyzing it during the simulation process can help us better understand the training situation. In contrast to global error, the consensus error is represented in Fig.7. Under the same return, the error becomes smaller and more steady, and the volatility tends to 0 over time. As the returns grow, the inaccuracy decreases and the oscillation amplitude decreases: when the return is -1000, the error curves are jagged. However, when the return is roughly -35, the curves are smooth and tend to 0 at the end of the time, indicating that the proposed controller can not only formulate policies to bring the system to consensus, but also to keep it stable.

Unlike the other parameters, the energy consumption in Fig.8 does not fully depict how the controller learns. However, we can see that the energy consumption decreases over time, which is likely due to the fact that the controller can only control the system to varying degrees without divergence until it fully learns the information. The loss is proportional to the system's initial state, if

the initial states are close to the consensus states, energy consumption will always be low. Nonetheless, it can be seen that when the return is about -35, the energy consumption will eventually converge to 0, indicating that the system has reached consensus with minimal energy consumption.

6 Conclusion

In this paper, a type of model-free leader-following multi-agent consensus problem under the directed topology is discussed, and a data-driven optimal consensus control method is proposed based on DDPG. The proposed method is used to assess the value of state-action pairs and generate control policies. After a period of training, the system can reach consensus with minimal energy consumption. Our method can avoid complex system modeling and observer construction while also solving the problem of dimensional explosion in state and action space, and doesn't need any initial admissible policies, giving it a broad application promise. Then the convergence and stability analysis are given, which provides sufficient theoretical support. Finally, some simulations are given to show the effectiveness of the proposed method. In the future, we will investigate how to apply this method to the nonlinear multi-agent systems.

This work was supported by the Tianjin Natural Science Foundation of China (20JCYBJC01060), the National Natural Science Foundation of China (62103203, 61973175), and the Fundamental Research Funds for the Central Universities, Nankai University(63221218).

Conflict of interest The authors declare that they have no conflict of interest.

- 1 Su H S, Zhang J X, Zeng Z G. Formation-containment control of multi-robot systems under a stochastic sampling mechanism. *Sci. China Tech. Sci.*, 2020, 63(6): 1025-1034.
- 2 Li Z, Yu H, Zhang G, et al. Network-wide traffic signal control optimization using a multi-agent deep reinforcement learning. *Transp Res Part C Emerg Technol.*, 2021, 125: 103059-103075.
- 3 Waschneck B, Reichstaller A, Belzner L, et al. Optimization of global production scheduling with deep reinforcement learning. *Procedia Cirp.*, 2018, 72: 1264-1269.
- 4 Cui K, Koepl H. Approximately solving mean field games via entropy-regularized deep reinforcement learning. *PMLR*, 2021: 1909-1917.
- 5 Lei L, Tan Y, Zheng K, et al. Deep reinforcement learning for autonomous internet of things: Model, applications and challenges. *IEEE Commun. Surv. Tutor.*, 2020, 22(3): 1722-1760.
- 6 Difilippo G, Fanti M P, Mangini A M. Maximizing Convergence Speed for Second Order Consensus in Leaderless Multi-Agent Systems. *IEEE/CAA J. Autom. Sinica*, 2021, 9(2):

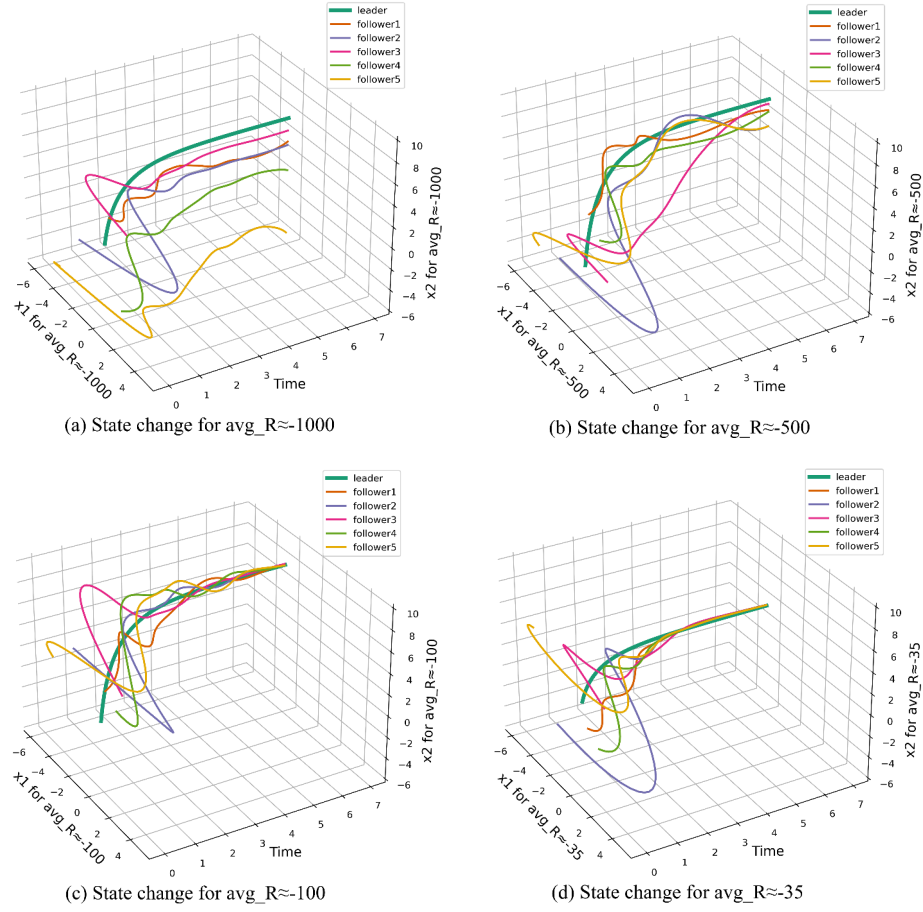


Figure 5 The graph of the state evolution under 4 different rewards

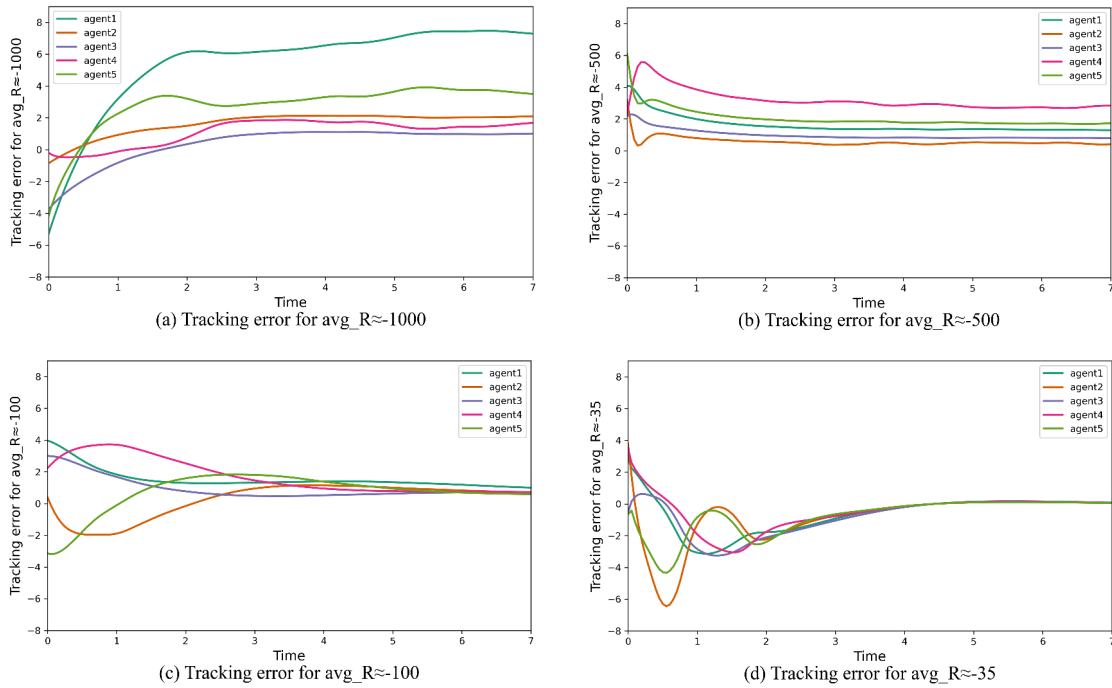


Figure 6 The graph of the tracking error evolution of the following agents under 4 different rewards

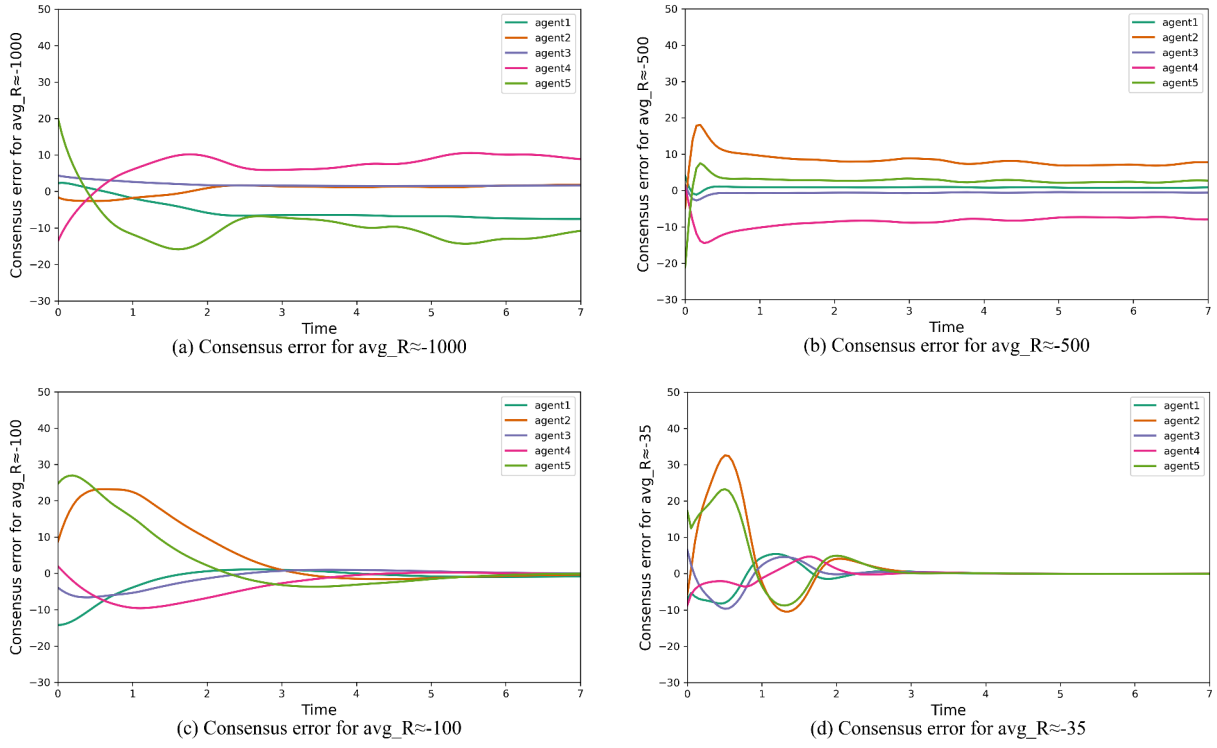


Figure 7 The graph of the consensus error evolution of the following agents under 4 different rewards

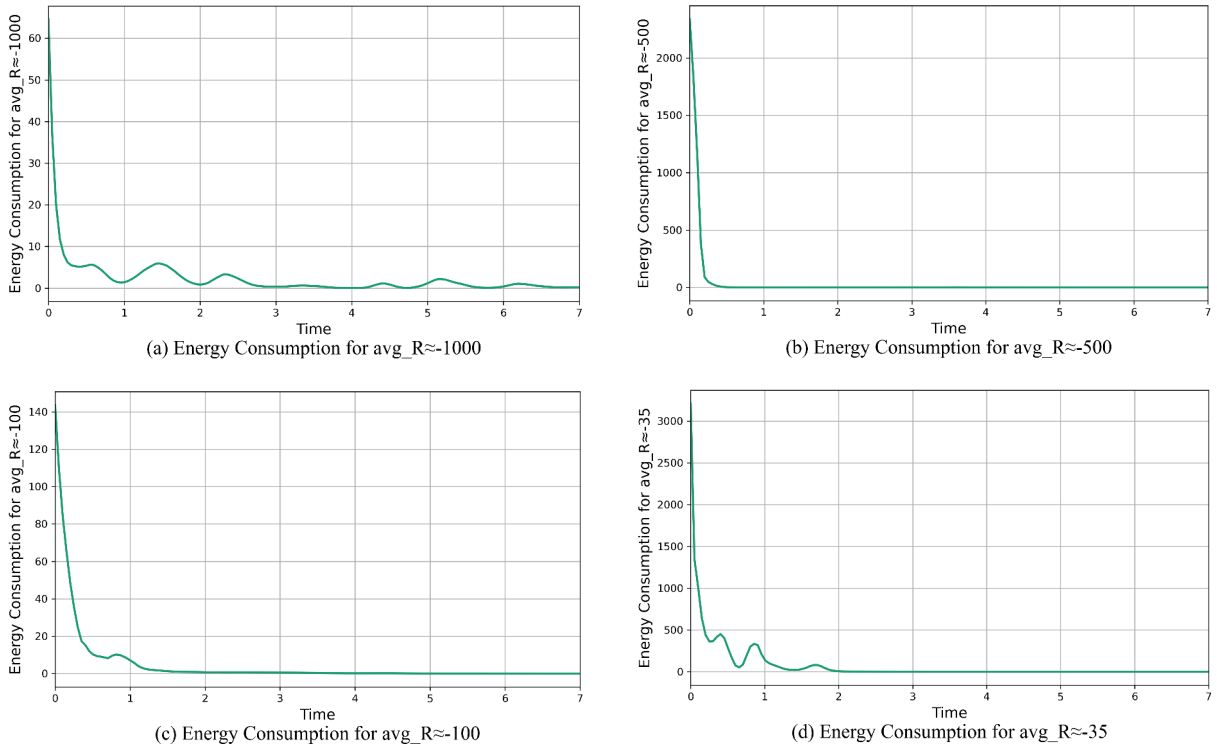


Figure 8 The graph of the energy consumption evolution of the following agents under 4 different rewards

- 259-269.
- 7 Yu W, Chen G, Cao M. Some necessary and sufficient conditions for second-order consensus in multi-agent dynamical systems. *Automatica*, 2010, 46(6): 1089-1095.
- 8 Ma L, Wang Z, Han Q L, et al. Consensus control of stochastic multi-agent systems: a survey. *Sci. China Inf. Sci*, 2017, 60(12): 1-15.
- 9 Wei Q, Wang X, Zhong X, et al. Consensus control of leader-following multi-agent systems in directed topology with heterogeneous disturbances. *IEEE/CAA J. Autom. Sinica*, 2021, 8(2): 423-431.
- 10 Cai Y, Zhang H, Zhang J, et al. Fixed-time leader-following/containment consensus for a class of nonlinear multi-agent systems. *Inf. Sci*, 2021, 555: 58-84.
- 11 Wang H, Xue B, Xue A. Leader-following consensus control for semi-Markov jump multi-agent systems: An adaptive event-triggered scheme. *J Franklin Inst*, 2021, 358(1): 428-447.
- 12 Wang X X, Liu Z X, Chen Z Q. Event-triggered fault-tolerant consensus control with control allocation in leader-following multi-agent systems. *Sci. China Tech. Sci*, 2021, 64(4): 879-889.
- 13 Zhu W, Jiang Z P. Event-based leader-following consensus of multi-agent systems with input time delay. *IEEE Trans. Automat. Contr*, 2014, 60(5): 1362-1367.
- 14 Sardellitti S, Barbarossa S, Swami A. Optimal topology control and power allocation for minimum energy consumption in consensus networks. *IEEE Trans. Signal Process*, 2011, 60(1): 383-399.
- 15 Li Q, Wei J, Gou Q, et al. Distributed adaptive fixed-time formation control for second-order multi-agent systems with collision avoidance. *Inf. Sci*, 2021, 564: 27-44.
- 16 He X Y, Wang Q Y, Hao Y Q. Finite-time adaptive formation control for multi-agent systems with uncertainties under collision avoidance and connectivity maintenance. *Sci. China Tech. Sci*, 2020, 63(11): 2305-2314.
- 17 Gronauer S, Diepold K. Multi-agent deep reinforcement learning: a survey. *Artif Intell Rev*, 2022, 55(2): 895-943.
- 18 Jiang R, Wang Z, He B, et al. A data-efficient goal-directed deep reinforcement learning method for robot visuomotor skill. *Neurocomputing*, 2021, 462: 389-401.
- 19 Zhang Y, Meng F, Li P, et al. MS-Ranker: Accumulating evidence from potentially correct candidates via reinforcement learning for answer selection. *Neurocomputing*, 2021, 449: 270-279.
- 20 Werbos P J, Miller W T, Sutton R S. A menu of designs for reinforcement learning over time. *Neural Netw. Contr*, 1990, 3: 67-95.
- 21 Doya K. Reinforcement learning in continuous time and space. *Neural Comput*, 2000, 12(1): 219-245.
- 22 Modares H, Lewis F L. Linear quadratic tracking control of partially-unknown continuous-time systems using reinforcement learning. *IEEE Trans. Automat. Contr*, 2014, 59(11): 3051-3056.
- 23 Modares H, Lewis F L. Optimal tracking control of nonlinear partially-unknown constrained-input systems using integral reinforcement learning. *Automatica*, 2014, 50(7): 1780-1792.
- 24 Luo B, Wu H N, Huang T, et al. Reinforcement learning solution for HJB equation arising in constrained optimal control problem. *Neural Netw*, 2015, 71: 150-158.
- 25 Fujita T, Ushio T. Reinforcement learning-based optimal control considering L computation time delay of linear discrete-time systems. *IEEE Symp. Adapt. Dyn. Program. Reinf. Learn.* 2014: 1-6.
- 26 Kiumarsi B, Lewis F L, Modares H, et al. Reinforcement Q-learning for optimal tracking control of linear discrete-time systems with unknown dynamics. *Automatica*, 2014, 50(4): 1167-1175.
- 27 Zhang X, Liu Y, Xu X, et al. Structural relational inference actor-critic for multi-agent reinforcement learning. *Neurocomputing*, 2021, 459: 383-394.
- 28 Li H, Liu D, Wang D. Integral reinforcement learning for linear continuous-time zero-sum games with completely unknown dynamics. *IEEE Trans. Autom. Sci. Eng*, 2014, 11(3): 706-714.
- 29 Vamvoudakis K G, Lewis F L, Hudak G R. Multi-agent differential graphical games: Online adaptive learning solution for synchronization with optimality. *Automatica*, 2012, 48(8): 1598-1611.
- 30 Abouheaf M I, Lewis F L, Vamvoudakis K G, et al. Multi-agent discrete-time graphical games and reinforcement learning solutions. *Automatica*, 2014, 50(12): 3038-3053.
- 31 Abouheaf M, Lewis F, Haesaert S, et al. Multi-agent discrete-time graphical games: interactive Nash equilibrium and value iteration solution. *ACC*, 2013: 4189-4195.
- 32 Chen C L P, Wen G X, Liu Y J, et al. Adaptive consensus control for a class of nonlinear multiagent time-delay systems using neural networks. *IEEE Trans Neural Netw Learn Syst*, 2014, 25(6): 1217-1226.
- 33 Li Y, Wang F, Liu Z, et al. Leader-Follower Optimal Consensus of Discrete-Time Linear Multi-agent Systems based on Q-Learning. CISC. Springer, Singapore, 2022: 492-501.
- 34 Zhu Z, Wang F, Liu Z, et al. Consensus of discrete-time multi-agent system based on Q-learning. *Control. Theory. and Appl*, 2021, 38(07): 997-1005.
- 35 Mu C, Zhao Q, Gao Z, et al. Q-learning solution for optimal consensus control of discrete-time multi agent systems using reinforcement learning. *J Franklin Inst*, 2019, 356(13): 6946-6967.
- 36 Zou W, Zhou C, Guo J, et al. Global adaptive leader-following consensus for second-order nonlinear multiagent systems with switching topologies. *IEEE Trans. Circuits Syst. II Express Briefs*, 2020, 68(2): 702-706.
- 37 Lillicrap T P, Hunt J J, Pritzel A, et al. Continuous control with deep reinforcement learning. *arXiv preprint*, 2015, 1509.02971).
- 38 Zhang H, Jiang H, Luo Y, et al. Data-driven optimal consensus control for discrete-time multi-agent systems with unknown dynamics using reinforcement learning method. *IEEE Trans. Ind. Electron*, 2016, 64(5): 4091-4100.
- 39 Abouheaf M I, Lewis F L, Mahmoud M S, et al. Discrete-time dynamic graphical games: model-free reinforcement learning solution. *Control. Theory Technol*, 2015, 13(1): 55-69.
- 40 Schaul T, Quan J, Antonoglou I, et al. Prioritized experience replay. *arXiv preprint*, 2015, 1511.05952).
- 41 Lazaric A, Restelli M, Bonarini A. Reinforcement learning in continuous action spaces through sequential monte carlo methods. *Proc. Adv. Neural Inf. Process. Syst*, 2007, 20: 1-8.